Free AWS Certified Developer – Associate Exam Practice

Explore realistic AWS Certified Developer Associate exam questions that cover key topics like serverless design, messaging services, security best practices, deployment strategies, and troubleshooting. This lesson helps you prepare by applying AWS development principles and services through scenario-based practice, improving your confidence for the DVA-C02 certification exam.

Question 1

A developer is building a stateless REST API using AWS Lambda as the backend for Amazon API Gateway. The API must scale automatically and should not store any user session data between requests.

Which design principle *best* supports this requirement?

A. Use Lambda environment variables to store session state.

B. Store session data in the `/tmp` directory.

C. Design the API to be stateless and externalize state.

D. Enable Sticky Sessions at the API Gateway level.

Answer Explanation

Ask

Question 2

A developer needs to process messages asynchronously to decouple two microservices. Messages must be processed at least once, and ordering is *not* required.

Which AWS service is *most* appropriate?

A. Amazon SNS

B. Amazon SQS Standard

C. AWS Step Functions

D. Amazon EventBridge

Answer Explanation

Question 3

A developer is maintaining an order processing system where an Amazon SQS queue triggers an AWS Lambda function. During peak hours, the downstream legacy database used by the Lambda function frequently returns “Connection Limit Exceeded” errors, causing the Lambda function to fail. The developer must ensure that no orders are lost and that the system can recover from these transient errors without issue.

Which two actions should the developer take to improve the application’s resilience? **(Select any two options.)**

Ask

A. Configure Lambda reserved concurrency to match the database connection limit.

B. Configure a Dead-letter Queue (DLQ) on the SQS queue to capture failed messages.

C. Switch the API Gateway to use synchronous Lambda invocation.

D. Implement exponential backoff and jitter in the Lambda code for database connection attempts.

E. Increase the Lambda function's memory allocation to handle higher throughput.

Answer Explanation

Question 4

A developer is writing a Python AWS Lambda function that processes records from an Amazon Kinesis Data Stream. The function must ensure idempotent writes to DynamoDB so that retries do not create duplicate items.

Which code snippet correctly implements this requirement?

A:

```
table.put_item(
    Item=item
)
```

B:

```
table.put_item(
    Item=item,
    ConditionExpression="attribute_not_exists(id)"
)

```

Ask

C:

```
table.update_item(
    Key={"id": item["id"]},
    UpdateExpression="SET processed = :p",
    ExpressionAttributeValues={":p": True}
)

```

D:

```
table.put_item(
    Item=item,
    ReturnValues="ALL_OLD"
)
```

Answer Explanation

Question 5

A company is building a high-throughput event-driven architecture where thousands of IoT devices publish telemetry events every second. Events must be ingested, processed asynchronously, and routed to multiple downstream services. Each consumer processes data independently, and failures in one consumer must not impact others.

Which two design choices *best* satisfy these requirements? **(Select any two options.)**

A. Publish events to Amazon SNS and subscribe multiple Amazon SQS queues.

B. Publish events directly to a single Amazon SQS FIFO queue.

C. Use Amazon EventBridge with multiple rule targets.

Ask

D. Invoke downstream Lambda functions synchronously.

E. Store events in DynamoDB Streams and poll the stream.

Answer Explanation

Question 6

A company's security policy requires that database credentials used by an AWS Lambda function must be rotated every 30 days. The rotation process must be automated and require minimal custom code.

Which solution should the developer implement?

A. AWS Systems Manager Parameter Store (Standard)

B. AWS Certificate Manager (ACM)

C. AWS Secrets Manager

D. AWS Key Management Service (KMS)

Answer Explanation

Question 7

A developer is using an IAM role to upload a configuration file to an Amazon S3 bucket. The bucket is configured to use Server-Side Encryption with AWS KMS keys (SSE-KMS). The upload attempt fails with an Access Denied error. An investigation reveals that the IAM role currently has no inline or managed p attached.

Ask

Which *two* permissions are required? **(Select any two options.)**

A. s3:PutObject on the S3 bucket resource.

B. `kms:GenerateDataKey` on the AWS KMS key.

C. `kms:Decrypt` on the AWS KMS key.

D. `s3:ListBucket` on the S3 bucket resource.

E. `kms:CreateKey` on the AWS KMS key.

Answer Explanation

Question 8

A company runs a serverless application using Amazon API Gateway and AWS Lambda. Users authenticate using Amazon Cognito User Pools. After authentication, users must upload files directly to an Amazon S3 bucket. The company wants to ensure:

- No long-term AWS credentials are exposed.
- Each user can upload objects only to their own S3 prefix.
- Access follows the principle of least privilege.

Which *two* actions should the developer take to meet these requirements? **(Select any two options.)**

A. Use Amazon Cognito Identity Pools to exchange User Pool tokens for temporary AWS credentials.

B. Attach an S3 bucket policy that allows public write access with a condition on the object prefix.

Ask

C. Configure IAM roles mapped to Cognito Identity Pool identities with scoped S3 permissions.

D. Generate pre-signed S3 URLs using an IAM user's access keys.

E. Store temporary AWS credentials in encrypted Lambda environment variables.

Answer Explanation

Question 9

A developer is writing a Python AWS Lambda function that retrieves a database password from AWS Secrets Manager at runtime. The function must follow AWS security best practices.

Which code snippet correctly retrieves the secret securely?

A:

```
import boto3

client = boto3.client('secretsmanager')
secret = client.get_secret_value(SecretId='db-password')
password = secret['SecretString']
```

B:

```
import boto3

session = boto3.session.Session(
    aws_access_key_id='AKIA...',
    aws_secret_access_key='SECRET'
)
client = session.client('secretsmanager')
password = client.get_secret_value('db-password')
```

Ask

C:

```
import os
password = os.environ['DB_PASSWORD']
```

D:

```
import boto3

client = boto3.client('ssm')
password = client.get_parameter(
    Name='db-password',
    WithDecryption=True
)
```

Answer Explanation

Question 10

A company's security policy dictates that IAM roles should only be assumed from the company's static corporate office IP range. A developer needs to implement a solution that provides real-time alerts whenever an AssumeRole operation occurs from an IP address outside of this authorized range.

Which solution *best* meets this requirement with the *least* amount of operational overhead?

A. Create an AWS Config custom rule that evaluates the sourceIPAddress of all IAM entities every hour.

B. Enable VPC Flow Logs for all subnets and use an AWS Lambda function to parse logs for IAM authentication traffic.

C. Configure AWS CloudTrail to send logs to Amazon CloudWatch Logs ar create a Metric Filter with an alarm based on the source IP.

Ask

D. Use Amazon GuardDuty to monitor for unusual path findings and configure it to send all findings to an SNS topic.

Answer Explanation

Question 11

A company is deploying a serverless REST API using AWS SAM, API Gateway, and Lambda. The team wants to deploy the same template to dev, staging, and production environments while meeting the following requirements:

- Each environment must point to a different Lambda version.
- No code changes are allowed between environments.
- Rollbacks must be fast and safe.

Which *two* solutions *best* meet these requirements? **(Select any two options.**

A. Use Lambda aliases for each environment and reference them from API Gateway.

B. Create separate SAM templates for dev, staging, and prod.

C. Use API Gateway stage variables to dynamically reference Lambda aliases.

D. Deploy all environments using the `$LATEST` Lambda version.

E. Hardcode environment identifiers in Lambda environment variables.

Answer Explanation

Ask

Question 12

A Lambda-based microservice is deployed through a CI/CD pipeline. After a recent update, the deployment succeeds, but the application starts failing immediately in production. The team wants to gradually shift traffic to new versions and automatically roll back if errors exceed a threshold.

Which deployment strategy meets these requirements with the *least* operational overhead?

A. All-at-once deployment using AWS SAM

B. Canary deployment using AWS SAM `DeploymentPreference`

C. Blue/green deployment using separate API Gateway stages

D. Rolling deployment using multiple Lambda aliases

Answer Explanation

Question 13

A developer wants to deploy the same AWS SAM application to multiple environments. Each environment must use a different DynamoDB table name, without modifying Lambda code.

```
Parameters:
  EnvironmentName:
    Type: String

Resources:
  MyFunction:
    Type: AWS::Serverless::Function
    Properties:
      Runtime: python3.11
      Handler: app.handler
      Environment:
        Variables:
```

Ask

```
      TABLE_NAME: !Sub "orders-${EnvironmentName}"
```

Which command should the developer use to deploy this stack to the staging environment while ensuring the `EnvironmentName` parameter is correctly assigned?

A:

```
sam deploy
```

B:

```
sam deploy --parameter-overrides EnvironmentName=staging
```

C:

```
sam deploy --env staging
```

D:

```
sam deploy --stack-name staging
```

Answer Explanation

Question 14

A company has a suite of 20 AWS Lambda functions that all require a specific large numerical processing library to run. Currently, each developer includes the library in every function's deployment package, resulting in slow deployme times and reaching the total account storage limit for Lambda.

Ask

Which solution should the developer implement to centralize the library and minimize the deployment package size while incurring the *least* operational overhead?

A. Store the library in an Amazon S3 bucket and use the Lambda function code to download and import it at runtime.

B. Create a custom Lambda runtime that pre-installs the library and use it for all functions.

C. Package the library into a Lambda layer and configure each function to reference that layer.

D. Use Lambda extensions to load the library into the execution environment's memory during the initialization phase.

Answer Explanation

Question 15

A company is using AWS SAM to manage a serverless application. The developer needs to ensure that the Lambda memory allocation is 128 MB in the development environment and 1024 MB in the production environment. The solution must ensure that the same template is used for both environments to prevent configuration drift.

Which approach meets these requirements with the *least* operational overhead?

A. Create a CloudFormation mapping section in the template and use `!FindInMap` to select memory size based on an Environment parameter.

Ask

B. Define a parameter for memory size in the template and provide the value using the `--parameter-overrides` flag during deployment.

C. Use AWS Systems Manager Parameter Store to store memory values and retrieve them within the Lambda code at runtime.

D. Duplicate the `AWS::Serverless::Function` resource within the same template and use CloudFormation Conditions to deploy only one.

Answer Explanation

Question 16

A developer is investigating an intermittent error in a production AWS Lambda function. The function processes thousands of requests per minute, making it difficult to find specific error messages in the standard CloudWatch Logs stream. The developer needs to identify the top five most frequent error messages across all invocations from the last 24 hours.

Which solution should the developer use to achieve this with the *least* operational overhead?

A. Use AWS X-Ray to visualize the service map and identify nodes with high error rates.

B. Run a query in Amazon CloudWatch Logs Insights using the stats and count commands.

C. Export the log groups to Amazon S3 and run a search using Amazon Athena.

D. Stream the logs to an Amazon OpenSearch Service cluster and use Dashboards to visualize the results.

Ask

Answer Explanation

Question 17

A company uses an Amazon SNS topic to fan out order events to multiple microservices, each with its own Amazon SQS queue. Currently, all microservices receive every message, but the shipping service only needs to process messages where the status is ready_to_ship. This is causing unnecessary Lambda invocations and resulting in increased costs.

Which combination of actions should the developer take to optimize this architecture for the *least* amount of unnecessary processing? **(Select any two options.)**

A. Use Amazon SNS FIFO topics to ensure the shipping service receives messages in the correct order.

B. Define an SNS subscription filter policy for the shipping service's queue.

C. Increase the SQS visibility timeout to give the shipping service more time to ignore irrelevant messages.

D. Add message attributes to the messages published to the SNS topic.

E. Enable SQS long polling to reduce the frequency of empty responses from the queue.

Answer Explanation

Question 18

A developer notices that a compute-intensive AWS Lambda function is tak

Ask

seconds to complete when configured with 128 MB of memory. Monitoring shows

that the function only uses 60 MB of RAM at its peak. The developer increases the memory to 512 MB, and the execution time drops to 2.5 seconds.

Which statement explains why this performance improvement occurred?

A. Increasing memory triggers a higher provisioned concurrency level, reducing cold starts.

B. AWS Lambda allocates CPU power proportionally to the configured memory size.

C. The function was previously limited by the 512 MB ephemeral storage (`/tmp`) limit.

D. Increasing memory switches the Lambda function to use a Graviton2 (ARM64) architecture.

Answer Explanation

Question 19

A serverless application uses **Amazon API Gateway → AWS Lambda → Amazon DynamoDB**.Users report intermittent latency spikes during peak hours. CloudWatch metrics show:

- Lambda duration is stable.
- DynamoDB latency is low.
- API Gateway latency spikes to over 1.8 seconds.

The developer must identify the root cause of the latency without modifying application code.

Ask

Which two actions should the developer take? **(Select any two options.)**

A. Enable AWS X-Ray on both API Gateway and Lambda.

B. Enable detailed CloudWatch metrics for DynamoDB.

C. Analyze API Gateway `IntegrationLatency` and `Latency` metrics.

D. Increase Lambda memory allocation.

E. Enable CloudTrail data events for DynamoDB.

Answer Explanation

Question 20

A developer needs to instrument an AWS Lambda function for custom metrics without making explicit PutMetricData API calls. The team wants the metric to be automatically extracted by CloudWatch from logs using Embedded Metric Format (EMF).

The metric requirements:

- Namespace: `OrderService`
- Metric name: `OrdersProcessed`
- Unit: `Count`
- Dimensions: `Service`, `Environment`
- Must work in Lambda by writing EMF JSON to standard output

Which code snippet correctly implements this?

A:

Ask

```
import json, time

def lambda_handler(event, context):
```

```
    emf = {
        "_aws": {
            "Timestamp": int(time.time() * 1000),
            "CloudWatchMetrics": [
                {
                    "Namespace": "OrderService",
                    "Dimensions": [["Service", "Environment"]],
                    "Metrics": [{"Name": "OrdersProcessed", "Unit": "Cou
                }
            ],
        },
        "Service": "Checkout",
        "Environment": "Prod",
        "OrdersProcessed": 1,
    }
    print(json.dumps(emf))
    return {"ok": True}
```

B:

```
import boto3

def lambda_handler(event, context):
    cw = boto3.client("cloudwatch")
    cw.put_metric_data(
        Namespace="OrderService",
        MetricData=[{
            "MetricName": "OrdersProcessed",
            "Value": 1,
            "Unit": "Count"
        }]
    )
    return {"ok": True}
```

C:

```
import json

def lambda_handler(event, context):
    print(json.dumps({
```

```
        "Namespace": "OrderService",
        "MetricName": "OrdersProcessed",
        "Value": 1,
        "Unit": "Count",
        "Service": "Checkout",
        "Environment": "Prod"
    }))
    return {"ok": True}
```

D:

Ask